

Scenario

Your manager, Tony, received 2 memory files from another facility within the ISAC. These 2 memory files were from actual incidents that took place within their facility a few years ago. Tony wants you to analyze them to see if you are able to analyze them for any signs of code injection and/or a rootkit to prepare you to detect APT attacks.

Goals

- Find the code injection and/or rootkits in the memory images.

Recommended tools

- Mandiant Redline
- Volatility Framework

Tasks

Task 1. Create a MANs file of APTA.VMEM.

- Run Redline and create a Mandiant Analysis File called APTA.

Task 2. Analyze the MANS file from task 1.

- Look for indicators of code injection using Redline.
 - Check if anything is Redlined? Do any of the process look suspicious?
 - Look for any indicators of code injection. Look for signs of a rootkit.

Task 3. Analyze APTA.VMEM within Volatility.

- Look more indicators of code injection in the memory image with Volatility.
 - Determine the profile of the memory image.
 - Investigate the processes. Do you see any processes that you didn't see in Redline?
 - Check the network connections. Any suspicious traffic? If so, what PID is associated with it?
 - Using Volatility, use the plugins to determine if code injection is within this process of svchost.exe.
 - Were any APIs hooked within this process?

Task 4. Create a MANs file of APTB.VMEM.

- Run Redline and create a Mandiant Analysis File called APTB.

Task 5. Analyze the MANS file from task 4.

- Look for signs of a rootkit using Redline.
 - Check for any indicators of code injection.
 - Check for any indicators of a rootkit.

Task 6. Analyze APTB.VMEM within Volatility.

- Look for indicators of a rootkit in the memory image with Volatility.
 - Determine the profile of the memory image.
 - Using Volatility plugins, check for any indicators of a rootkit.
 - What is the suspicious driver from the output?
 - Obtain the base address of the module and dump the malicious driver.
- Look for this malicious driver within Redline.
 - Would you have spotted it without Volatility?
 - Can you obtain the base address for the module from Redline as well?

SOLUTIONS

Below, you can find solutions for each task. Remember, though, that you can follow your own strategy, which may be different from the one explained in the following lab.

Task 1: Create a MANs file of APTA.VMEM.

1. Open Redline and select **From a Saved Memory File** under **Analyze Data**.
1. Under Location of **Saved Memory Image**, click **Browse**.
1. Change **Memory Images** to **All Files (*)**.
1. Choose **apta.vmem** and choose **Next**.
2. Click **Edit your script** select **Strings** for both **Process Listing** and **Drivers Enumeration** then choose **Ok**.
3. Change the **Name** to **APTA** and choose **Ok**.
4. If UAC is enabled, select **Yes**.

You should see Redline creating the analysis of the memory file, as shown below.

Task 2: Analyze the MANS file from task 1.

1. After analysis is complete, we can look at the processes by selecting **Processes** under **Analysis Data**.

As we can see nothing is redlined. Based on what we know about Windows processes we can attempt to see if anything is odd (wrong parent process, wrong path, etc.). To do this we can switch to **Hierarchical Processes** view.

1. Click on **Hierarchical Processes** view under **Analysis Data**.

Do we see anything out of the ordinary? If not then let's look at **Memory Sections** for signs of code injection.

1. Click on **Memory Sections** (submenu item under **Processes**) under **Analysis Data**.
2. Click on **Injected Memory Sections** under **Filters**.

We should see 24 items listed as injected with code and by glancing at the results it appears to be nearly every processes within the memory image.

We can look at **Strings** and/or **Ports** but instead we'll look at **Hooks** to see any indicators of some sort of rootkit.

1. Click on **Hooks** under **Analysis Data**.
2. Click on **SSDT Hooks** under **Filters**.
3. Click on **IDT Hooks** under **Filters**.
4. Click on **Untrusted Hooks** under **Filters**.

We didn't see anything under SSDT and IDT Hooks but we're seeing some entries under Untrusted Hooks. At this point, with the data gathered, we definitely know there was code injection in many processes. We can now take a closer look at things in Volatility.

Task 3: Analyze APTA.VMEM within Volatility.

Copy paste the apta.vmem on desktop

```
# We will need to determine what profile to use for this memory image file within Volatility. Type the following within CMD or PowerShell. Volatility is being called from within the its folder and the apta.vmem is on the desktop. That is why it's calling the VMEM file with ..\apta.vmem.
PS>python vol.py -f C:\Users\Administrator\Desktop\apta.vmem imageinfo
```

We can see from the output that the profile is **WinXPSP2x86**.

```
# We'll begin by looking at the processes with Volatility using PSLIST.
PS>python vol.py -f C:\Users\Administrator\Desktop\apta.vmem --profile=WinXPSP2x86 pslist
```

If we compare Redline and Volatility we should see 1 process, cmd.exe (124), that we didn't see in Redline.

```
# Next we can take a look at the processes with Volatility using PSSCAN.
PS>python vol.py -f C:\Users\Administrator\Desktop\apta.vmem --profile=WinXPSP2x86 psscan
```

If we compare Redline and Volatility we should see another process, VMip.exe (1944), that we didn't see in Redline.

This process looks like it's a child process to cmd.exe (124).

We can use the Volatility **PSTREE** plugin to get a hierarchical process.

```
PS>python vol.py -f C:\Users\Administrator\Desktop\apta.vmem --profile=WinXPSP2x86 pstree
```

From the output we can't see VMip.exe (1944) which was spawned by cmd.exe (124) but we can see cmd.exe (124).

We can use **PSXVIEW** to see if this process is showing indicators that it's a hidden process.

```
PS>python vol.py -f C:\Users\Administrator\Desktop\apta.vem --profile=WinXPSP2x86 psxview
```

By the output from **PSXVIEW** we see that VMip.exe (1944) shows indicators of a hidden process, along with cmd.exe (124). Remember that we didn't see these 2 processes in Redline but does this mean that these 2 processes are malicious? Remember that Redline indicated many processes were injected except these 2 processes because they were not visible in Redline. Let's see if VMip.exe is trying to connect to any remote IP addresses by using **CONNSCAN** (Windows XP).

```
PS>python vol.py -f C:\Users\Administrator\Desktop\apta.vmem --profile=WinXPSP2x86 connscan
```

From the output we can see that process 856 is made outbound connections on ports 1054 & 1056 to an IP on port 80. What is process 856? The answer is svchost.exe.

This process is one of the processes shown as injected. Let's us **MALFIND** to further examine.

```
PS>python vol.py -f C:\Users\Administrator\Desktop\apta.vmem --profile=WinXPSP2x86 malfind -p 856
```

Based on the output we can see 1 memory section containing an MZ header which is an indicator of code injection.

Let's see if this process is hooking to any APIs with **APIHOOKS**.

```
PS>python vol.py -f C:\Users\Administrator\Desktop\apta.vmem --profile=WinXPSP2x86 apihooks -p 856
```

From the output we see 2 APIs are hooked (**NtCreateThread** and **ZwCreateThread**). **Unknown** being shown as the **Hooking module** adds to conclusion suspicious activity.

Note: We have only focused on process 856 but you can use these techniques with nearly all of the processes as they're all injected. The malware you just examined was **Zeus**.

[Task 4: Create a MANs file of APTB.VMEM.](#)

Follow the same steps from Task 1 but you're performing the steps against **APTB.VMEM** and naming the analysis file as **APT.B**.

Task 5. Analyze the MANS file from task 4.

- 1. Click on **Memory Sections** (submenu item under **Processes**) under **Analysis Data**.
- 2. Click on **Injected Memory Sections** under **Filters**.

We should see 1 item listed as injected with code and it has the same process as the previous memory image. Next we'll look at **Hooks** to see any indicators of some sort of rootkit.

- 1. Click on **Hooks** under **Analysis Data**.
- 2. Click on **SSDT Hooks** under **Filters**.
- 3. Click on **IDT Hooks** under **Filters**.
- 4. Click on **Untrusted Hooks** under **Filters**.

We didn't see anything under SSDT and IDT Hooks but we're seeing some entries under Untrusted Hooks. As before, we can now take a closer look at things in Volatility.

Task 6: Analyze APTB.VMEM within Volatility.

As before we will need to determine what profile to use for this memory image file within Volatility. Type the following within CMD or PowerShell. Volatility is being called from within the its folder and the aptb.vmem is on the desktop. That is why it's calling the VMEM file with ..\aptb.vmem.
PS>python vol.py -f C:\Users\Administrator\Desktop\aptb.vmem imageinfo

Since the goal is to look for a rootkit, we should only focus on the Volatility plugins for rootkit detection. Let's use **SSDT**.

```
PS>python vol.py -f C:\Users\Administrator\Desktop\aptb.vmem --profile=WinXPSP2x86 ssdt
```

From the output above we see some hooked functions pointing to the same driver, **00004A2A.sys**. Alternatively the **THREADS** plugin could be used to obtain the same conclusion about the malicious driver.

```
PS>python vol.py -f C:\Users\Administrator\Desktop\aptb.vmem --profile=WinXPSP2x86 threads -L
```

Next, we can extract this malcious driver for further analysis. This process starts with using **MODULES**.

```
PS>python vol.py -f C:\Users\Administrator\Desktop\aptb.vmem --profile=WinXPSP2x86 modules
```

We need to obtain the base address. The base address is **0xff0d1000**. Now we can use **MODDUMP** to extract the driver from the memory image.

```
PS>mkdir ..\APTB\Moddump
PS>python vol.py -f ..\aptb.vmem --profile=WinXPSP2x86 moddump -b 0xff0d1000 --dump-dir ..\APTB\Moddump
C:\volatility>vol.py -f C:\Users\Administrator\Desktop\aptb.vmem --profile=WinXPSP2x86 moddump -b 0xff0d1000 --dump-dir ..\APTB\Moddump
Volatility Foundation Volatility Framework 2.6
Module Base Module Name Result
-----
0x0ff0d1000 00004A2A OK: driver.ff0d1000.sys
```

Let's look for this malicious driver in Redline and see if it would have been easy to identify.

- 1. Open Redline and load **APTB.MANS** file.
- 2. Click on **Driver Modules** under **Analysis Data**.
- 3. Click on the **Name** column to sort the column contents.

Ntfs.sys	564 Kilobytes	0xfc296204	0xfc211000	0x810d6710
NDIS.sys	180 Kilobytes	0xfc20d205	0xfc1e4000	0x810d66a8
Mup.sys	108 Kilobytes	0xfc1e0bfa	0xfc1c9000	0x810d6640
agp440.sys	44 Kilobytes	0xfc4f3d85	0xfc4eb000	0x810d65d0
00004A2A	32.845 Kilobytes	0xfc759ec0	0xff0d1000	0xff375b08
\\??\C:\Program Files\VMware\VMware Tools\Drivers\memctl\vmemctl.sys	8 Kilobytes	0xfc9f85a8	0xfc9f7000	0x80ef7d70

As we can see its not the only driver without a full path but it is the only driver that is named with a number value. Also you can see the Module Base address which was obtained in Volatility using **MODULES** and extracted using **MODDUMP**.

Note: The malware you just analyzed was **Black Energy**.

Conclusion

After this lab you will be able to Use Redline to find evidence of code injection and/or rootkits, Use Volatility to find evidence of code injection and/or rootkits, Use Volatility to find information that was not seen in Redline.